

Frontend Development with React

Components • Hooks • Consuming the Capstone API



COMPONENTS



HOOKS



API

React Component Essentials

WHAT'S AHEAD

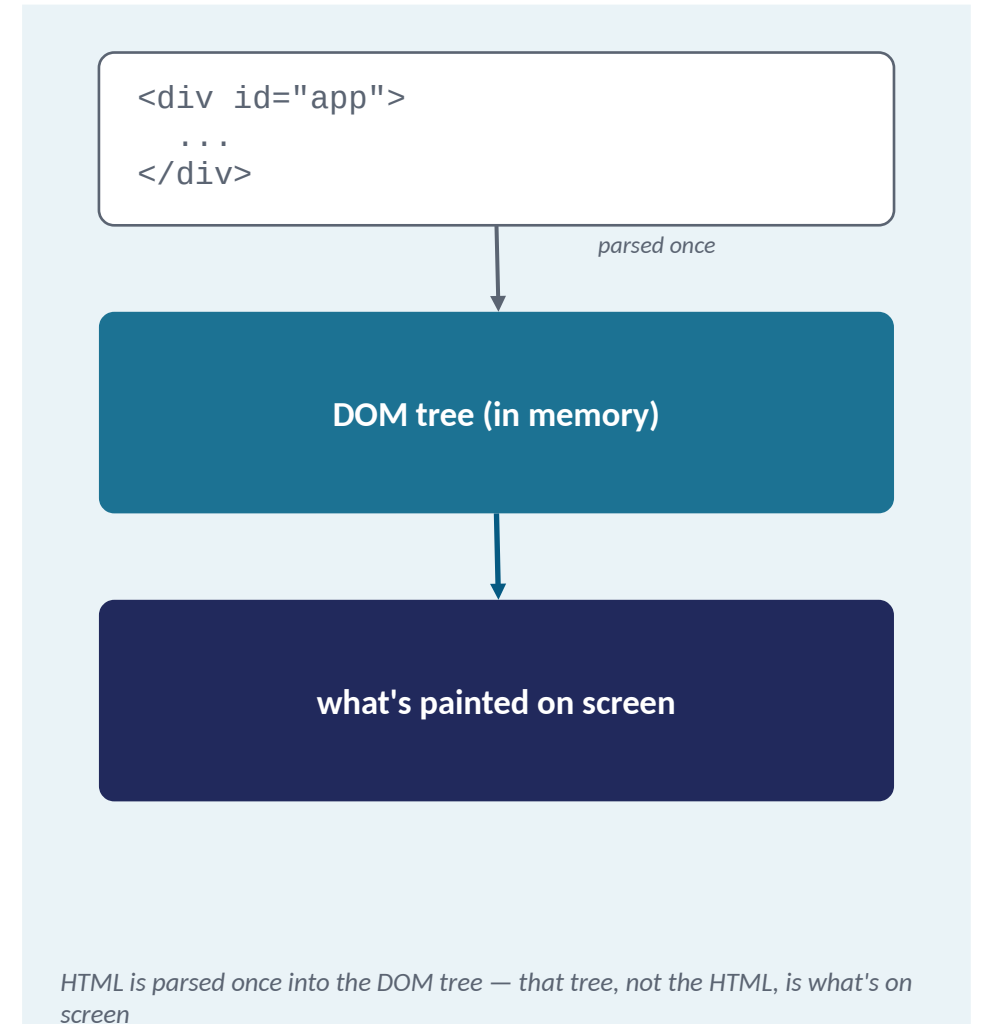
Why manual DOM updates break down, and React's declarative fix
JSX, the virtual DOM, and reconciliation — how React actually works

Components, props, and state

Hooks: why they exist, `useState` and `useEffect`

Where This Starts: The DOM Is on Screen

- A browser parses loaded HTML once into an in-memory tree — the DOM
- The DOM tree, not the original HTML file, is what's actually painted on screen
- A page that never changes after load can stop there
- Any later change — a counter, a new row, a toggled class — means mutating that live tree

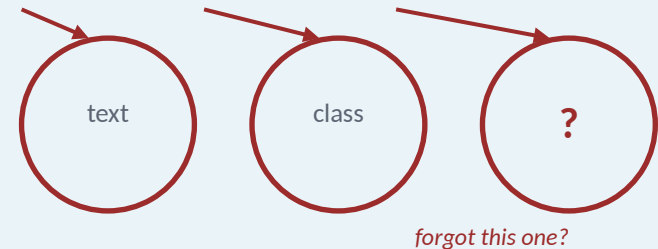


The Cost of Manual DOM Updates

PROBLEM

- Updating the page means finding the right element and mutating it, by hand, every time
- One piece of data can appear in several places — each needs its own update call
- Add one more dependent element and there are now two update functions to keep in sync
- On a real screen this bookkeeping is where bugs live, not the business logic

```
renderBookingCount(count) {  
  el.textContent = ...  
  el.classList...  
}
```



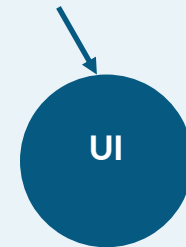
A JavaScript snippet that must correctly update every dependent page element by hand

Declarative Components

SOLUTION

- Describe what the UI should look like for the current data, not how to update it
- React re-runs the description and figures out what actually needs to change
- Same booking count, written as a function returning markup, not DOM edits
- The shift for the rest of today: "how do I update it" → "what should it look like now"

```
function BookingCount({ count }) {  
  return <span>{count} bookings</span>;  
}
```



One description, one clean update path — React handles the rest

JSX — Markup and Logic, Together

- JSX: HTML-like markup written inside a JavaScript file
- Keeps a component's rendering logic and markup in sync on every edit
- Three rules: one root element, every tag closed, camelCase attributes
- Needs a compiler (Babel) — browsers only run plain JavaScript, never JSX directly

```
<img src=... />
```

self-closed tag

```
className="empty"
```

camelCase attribute

```
{count} bookings
```

JS expression in { }

Three JSX rules, called out on a real snippet

Choosing Your Setup: Webpack vs. Vite, JS vs. TS

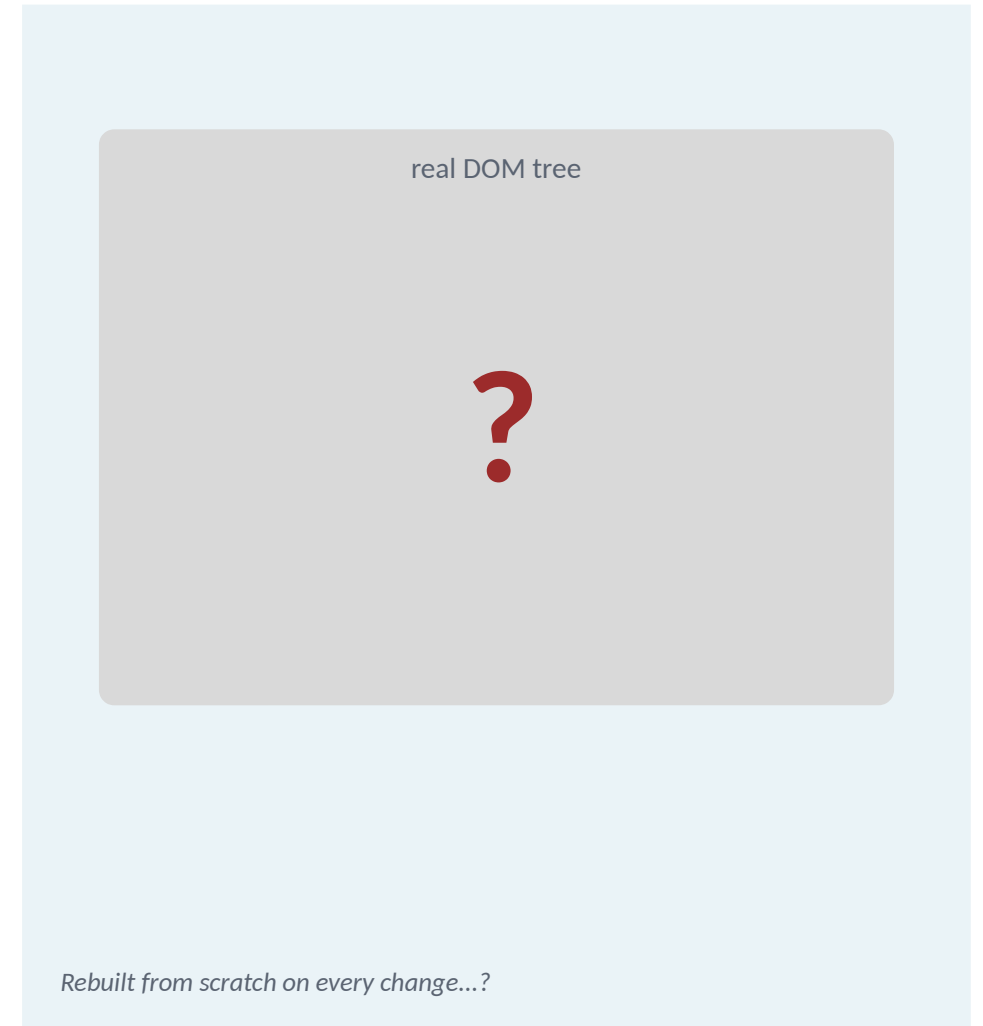
- Two independent choices: which bundler, and which language
- Webpack (Create React App's old default) processes the whole app upfront — slow as it grows
- Vite pre-bundles dependencies once, serves your own code on demand — near-instant start
- Four real starting points — pick one before today's hands-on



Is React Rebuilding the Whole Page Every Time?

PROBLEM

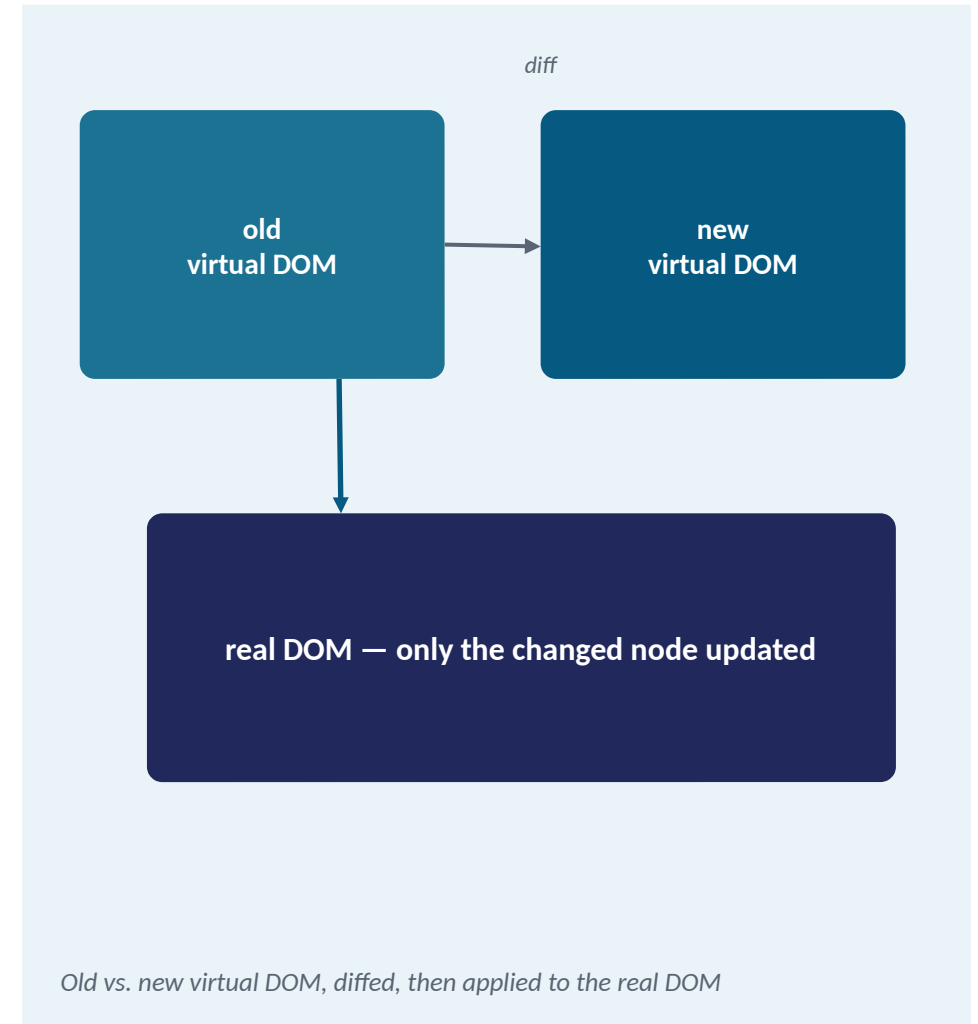
- BookingCount re-runs and returns a brand-new description every time count changes
- A brand-new description on every change sounds expensive, taken literally
- Rebuilding the real page from scratch on every update would be far too slow
- Something has to stand between "describe it all again" and "touch only what changed"



The Virtual DOM & Reconciliation

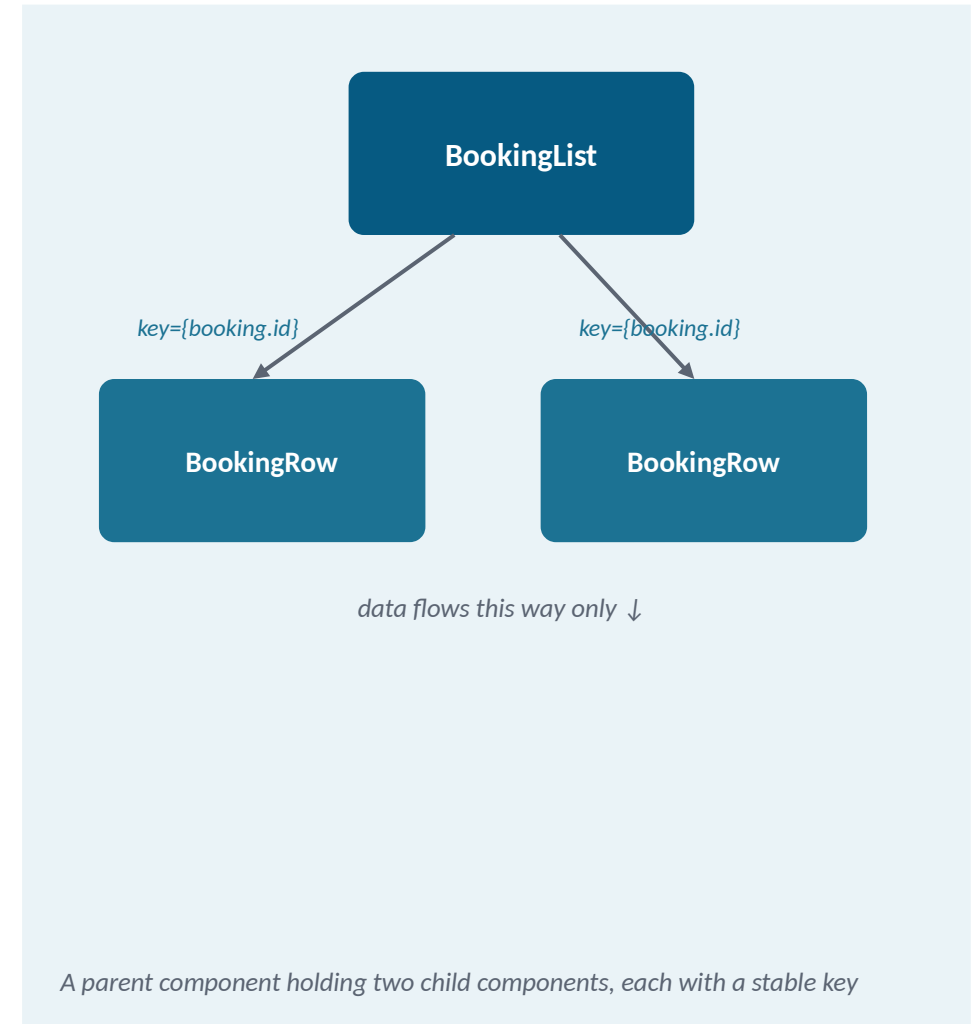
SOLUTION

- React keeps a lightweight, in-memory description of what the UI should look like
- On a change, React compares the new description against the previous one
- It works out the smallest set of real DOM edits needed, applies only those
- This comparison process is called reconciliation — React's actual efficiency mechanism



Components: The Unit This Gets Organized Into

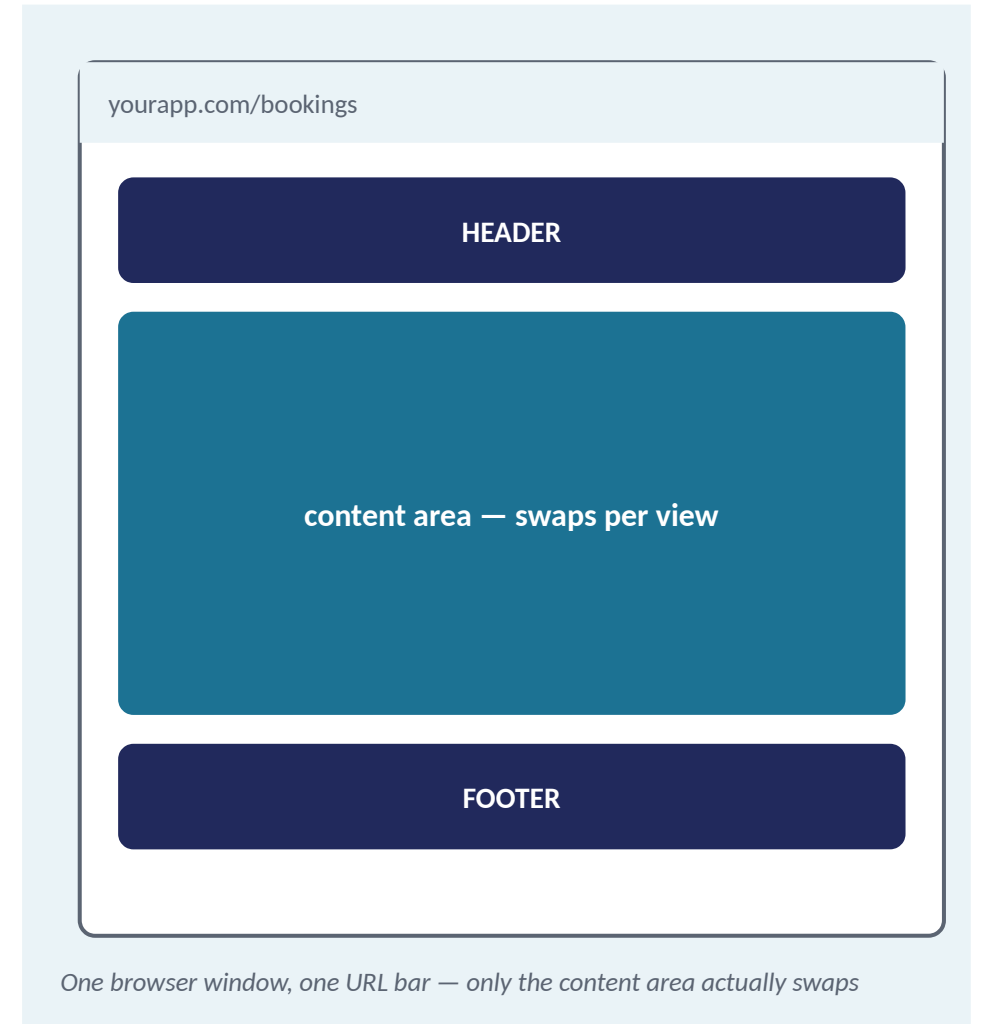
- A component is a JavaScript function, capitalized name, returning JSX markup
- Components compose: build small ones, nest and reuse them to build whole screens
- "Thinking in React" rule: one component, one concern — or split it up
- Data flows one way — parent to child — down the component hierarchy
- A list needs a stable key (its ID) — index keys break silently on reorder



HANDS-ON Scaffold a component tree for the capstone's UI using an AI coding assistant.

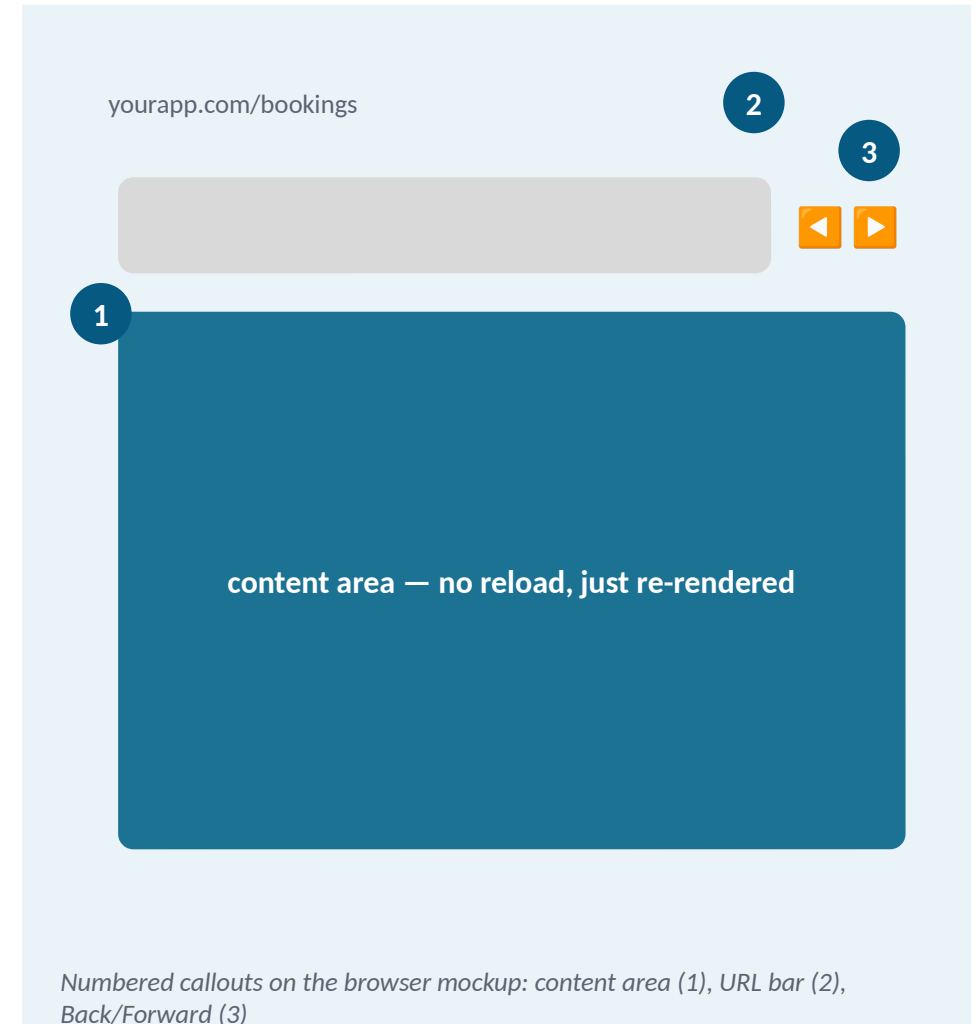
Putting It Together: What Makes This an SPA

- Only one real HTML document ever loads; JavaScript updates its content for each view
- That's the browser vendors' own definition of a Single Page Application (SPA)
- The same reconciliation already covered — applied across whole views, not one `<span>`
- Because only one page ever loads, an SPA has to deliberately recreate what a reload gave free



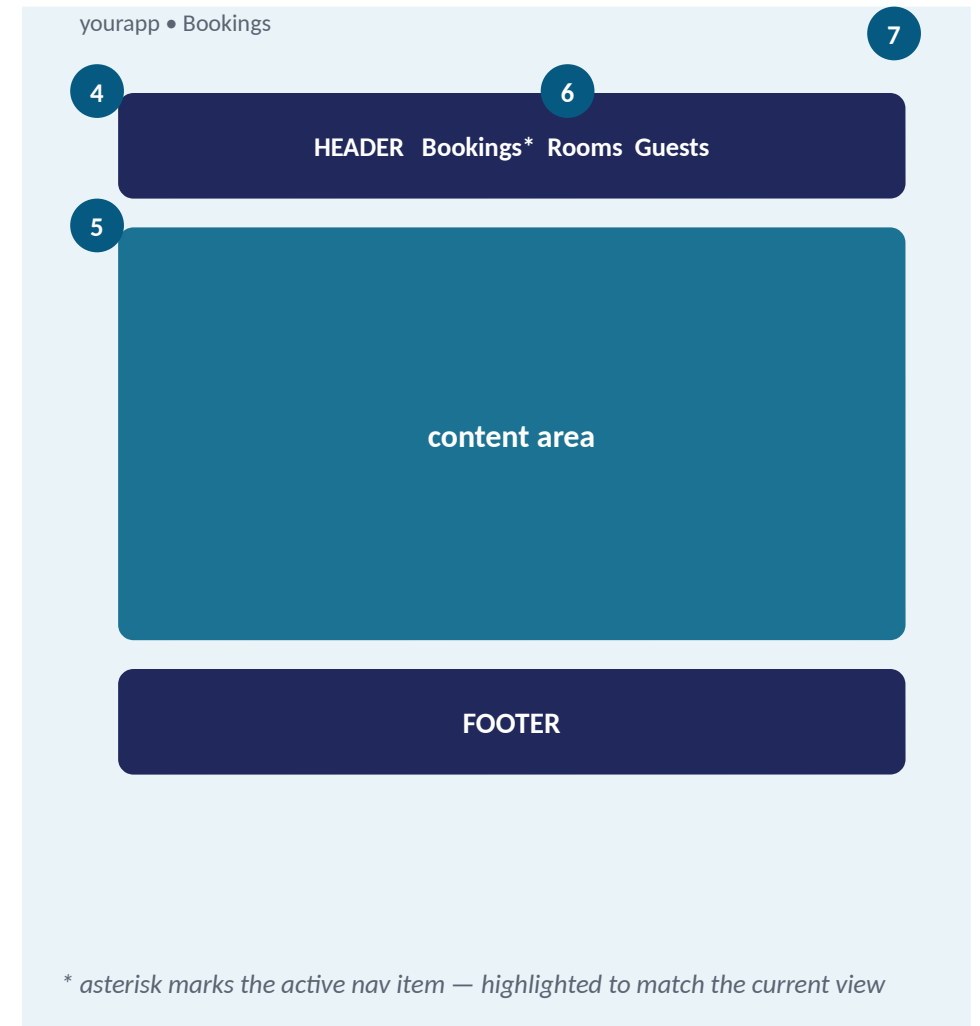
SPA Characteristics (1-3): Navigation Feels Seamless

- 1. The page never fully reloads when the user "navigates"
- 2. The URL still changes to match what's on screen
- 3. Back/Forward still work — history is maintained without a page refresh



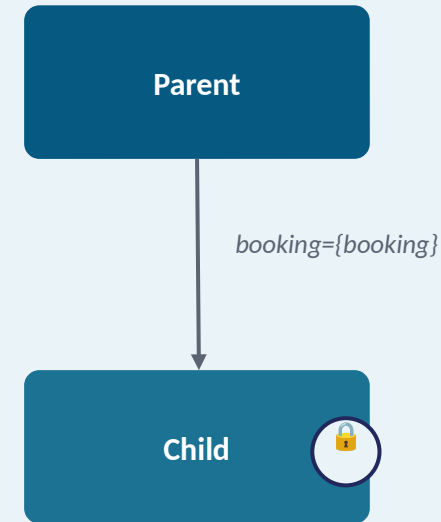
SPA Characteristics (4–7): The Shell Stays Put

- 4. Header and footer mount once and stay intact across every view
- 5. Only the section between header and footer actually updates
- 6. The active menu item visibly matches the current view
- 7. The browser tab's title updates for every view



Props — Passing Data Down

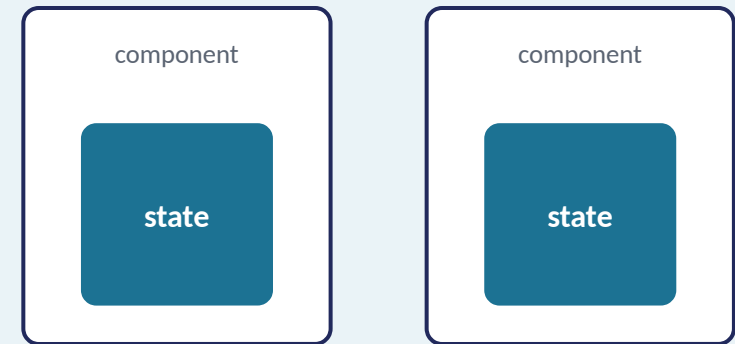
- Props: how a parent hands data to a child component
- Any JavaScript value — string, number, object, array, even a function
- Read as a single parameter, usually destructured: function
`BookingRow({ booking })`
- Read-only from the child's side — never mutate a prop directly



Props flow down; the child treats them as read-only

State — A Component's Own Memory

- State: what React calls a component's own private, remembered data
- Fully isolated per rendered copy — two copies never share state
- Answers "what do I remember about myself," not "what did my parent hand me"
- Needed for anything tracked locally: an open dropdown, typed text, an in-flight request
- Events (onClick, onChange) call the setter — pass a function, never call it immediately



onClick →

✓ `onClick={() => setIsOpen(true)}`

✗ `onClick={setIsOpen(true)}`

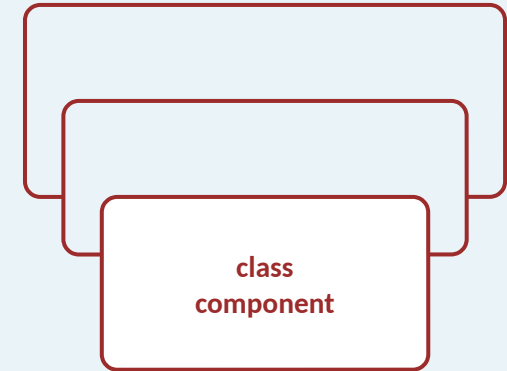
the second form runs the setter on every render, not on click

Two identical components, each with its own private notebook

Why Not Just Use a Class Component?

PROBLEM

- For years, only class components could hold state at all
- Reusing stateful logic between components needed render props / HOCs — "wrapper hell"
- A single lifecycle method often mixed unrelated logic just because both ran "on mount"
- Classes require understanding confusing this-binding, and minify/hot-reload poorly



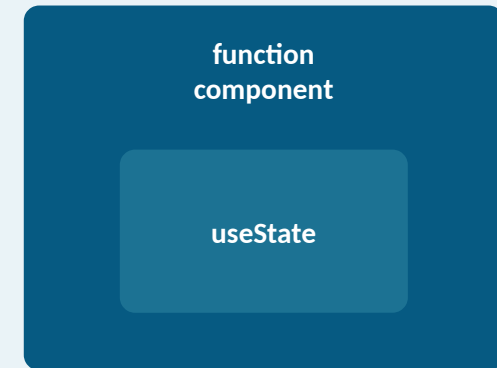
provider / HOC / render-prop layers

A class wrapped in layers of providers, HOCs, and render props

Hooks & useState

SOLUTION

- Hooks: special functions (always start with use) that let a plain function hold state
- `useState(initialValue)` returns exactly two things: the value, and a setter function
- Calling the setter tells React to remember the new value and re-render
- Hooks can only be called at the top level — never in a loop, condition, or nested function

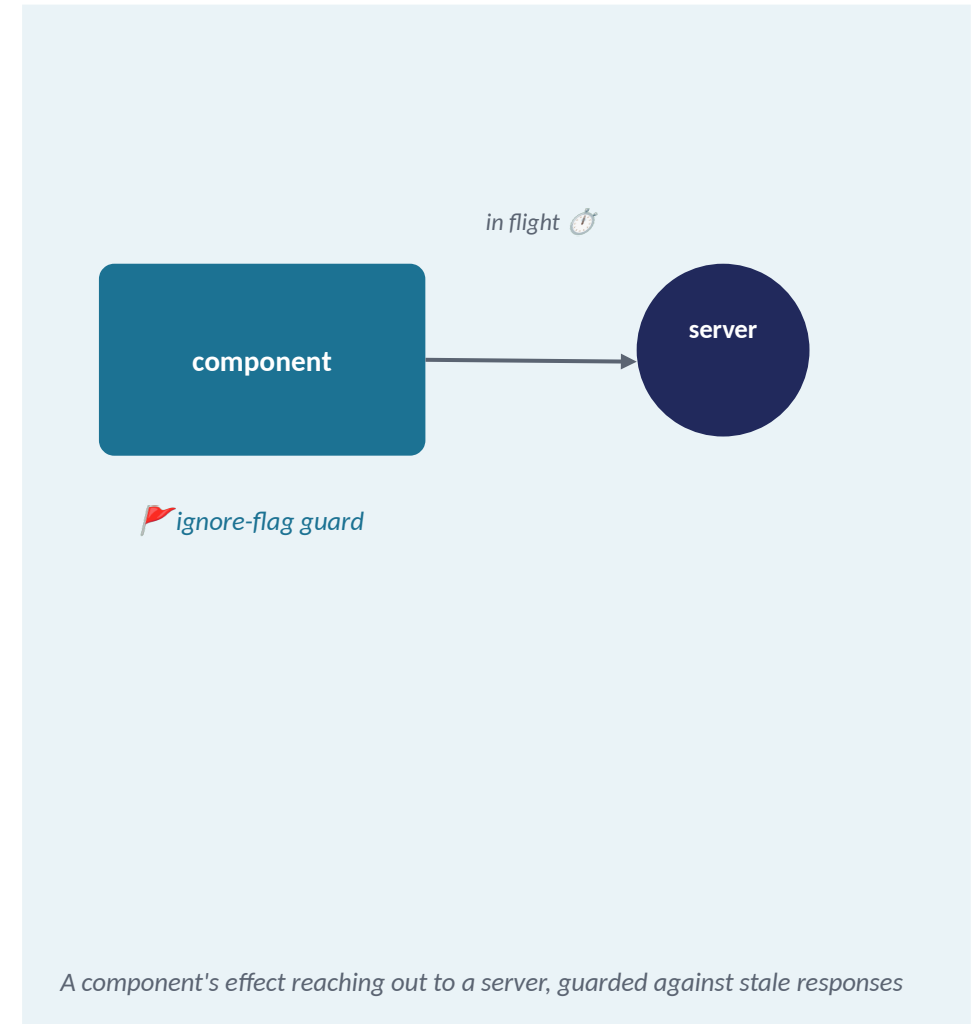


state lives here

One plain function, holding its own state directly — no wrapper layers

useEffect — Reaching Outside the Component

- useEffect synchronizes a component with something outside React — usually a network call
- An empty dependency array ([]) means "run once, when the component first appears"
- A returned cleanup function runs before the next effect, and once more on removal
- Fetching can resolve out of order (a race condition) — guard it with an ignore flag



HANDS-ON Implement state with `useState` and a data-fetching effect with `useEffect`.

Consuming the Capstone API

WHAT'S AHEAD

From a public practice endpoint to the real capstone API, with axios

Loading and error states — handling the gap before data arrives

Reading axios's error shape — server error vs. network failure

Reviewing AI-generated component code before it ships

From Practice API to Real Capstone API

- Same useState/useEffect mechanic from this morning — new target, that's all
- axios: a promise-based HTTP client, parses JSON automatically (response.data)
- Real base URL and endpoint from the team's own API contract, not a practice URL
- Real Authorization header — the request-header skill already in place
- An interceptor can attach that header to every request in one place, not each call site

HANDS-ON Connect the component to the capstone's REST API.

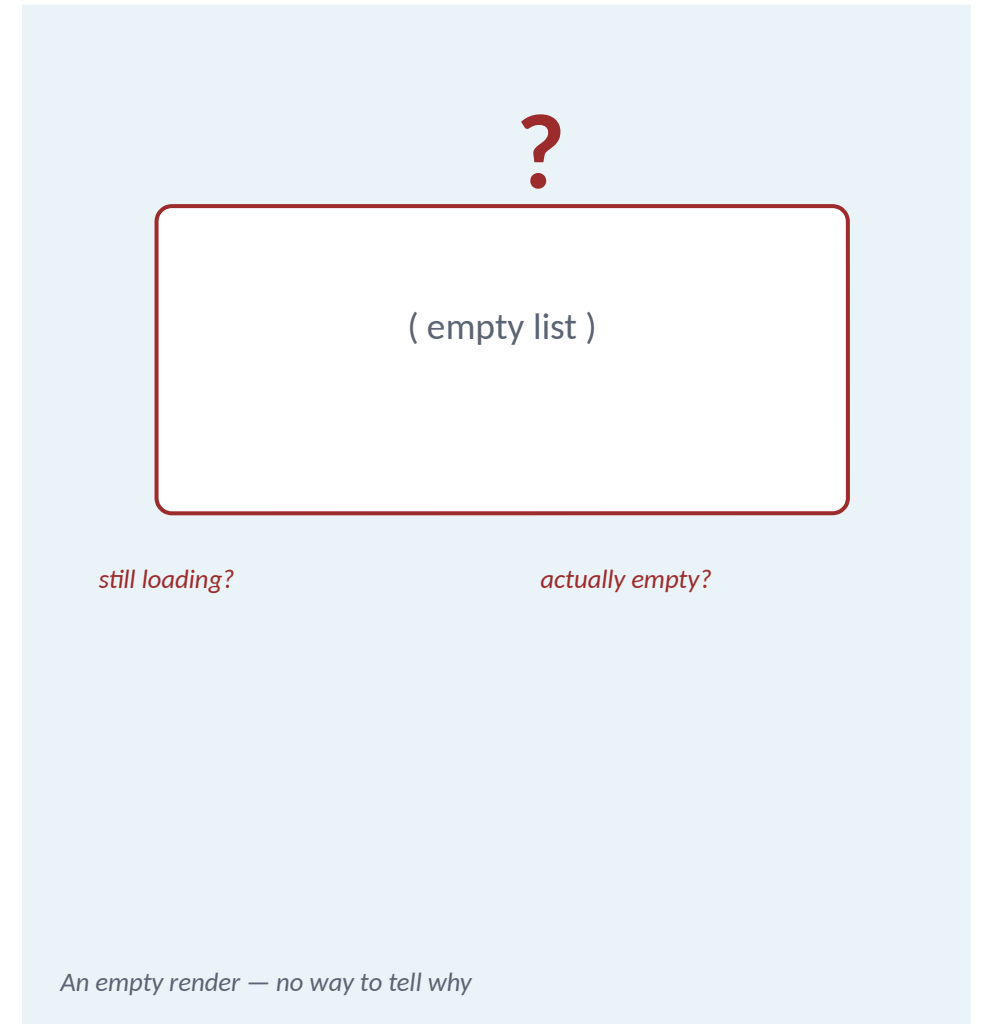


The same effect, now pointed at the real, authenticated capstone API

What Shows Before the Data Arrives?

PROBLEM

- Between render and response, the state holding the list is still whatever it started as
- A UI with no handling for that gap shows "no bookings" for a moment — or forever
- A failed request that's silently ignored looks identical to "there's nothing here"
- The fix needs to live in the same component's state, not a bolted-on workaround



Loading & Error States

SOLUTION

- Two more state values: isLoading (boolean) and error (whatever went wrong, if anything)
- Set isLoading true and clear error at the start of every request
- Render one of three things: a loading indicator, an error message, or the real data
- This "render one of several things" pattern has a name: conditional rendering



cond ? A : B

either/or

cond && A

only sometimes

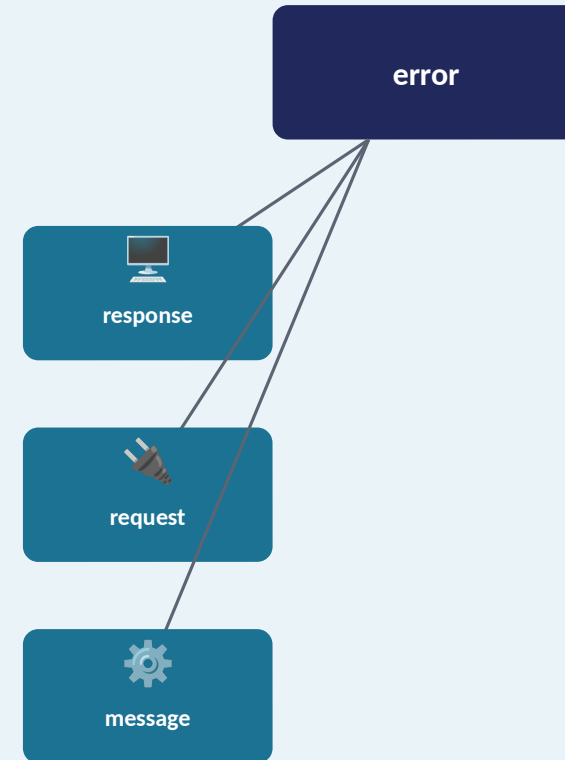
early return

skip the rest

Three explicit render states instead of one ambiguous empty list

Reading axios's Error Shape

- `error.response`: the server responded, just with a bad status (`.status`, `.data`)
- `error.request`: the request was sent, but nothing came back — a real network failure
- `error.message`: something failed just setting up the request itself
- Distinguishing these gives a more useful message than one generic "something went wrong"



One error object, three distinguishable failure paths

Reviewing the AI-Generated Code

- Read the diff line by line before accepting it — same discipline as prior AI-assisted work
- Confirm the ignore-flag guard is still present and checked before every setter call
- Confirm isLoading and error both reset at the start of the effect, not just one
- Confirm the URL and header match the team's real API contract, not an invented shape

```
+ let ignore = false;  
+ setIsLoading(true);  
+ setError(null);  
  axios.get(realUrl, {  
    headers: { Authorization }  
  })
```

✓ ignore flag

✓ both states reset

✓ real URL & header

A reviewed diff with three specific checks called out

Beyond Today's Essentials

WHAT'S AHEAD

React: a library, not a framework

Routing between views

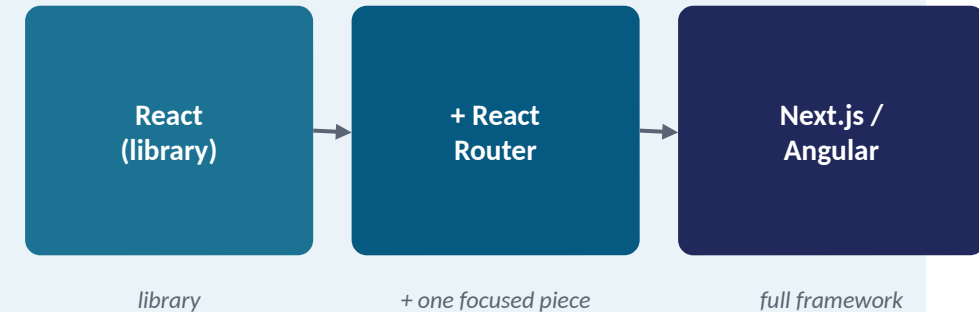
Writing data back — forms & REST CRUD

From npm run dev to a deployable build

Hands-on practice for these lives in today's lab guide

React Is a Library, Not a Framework

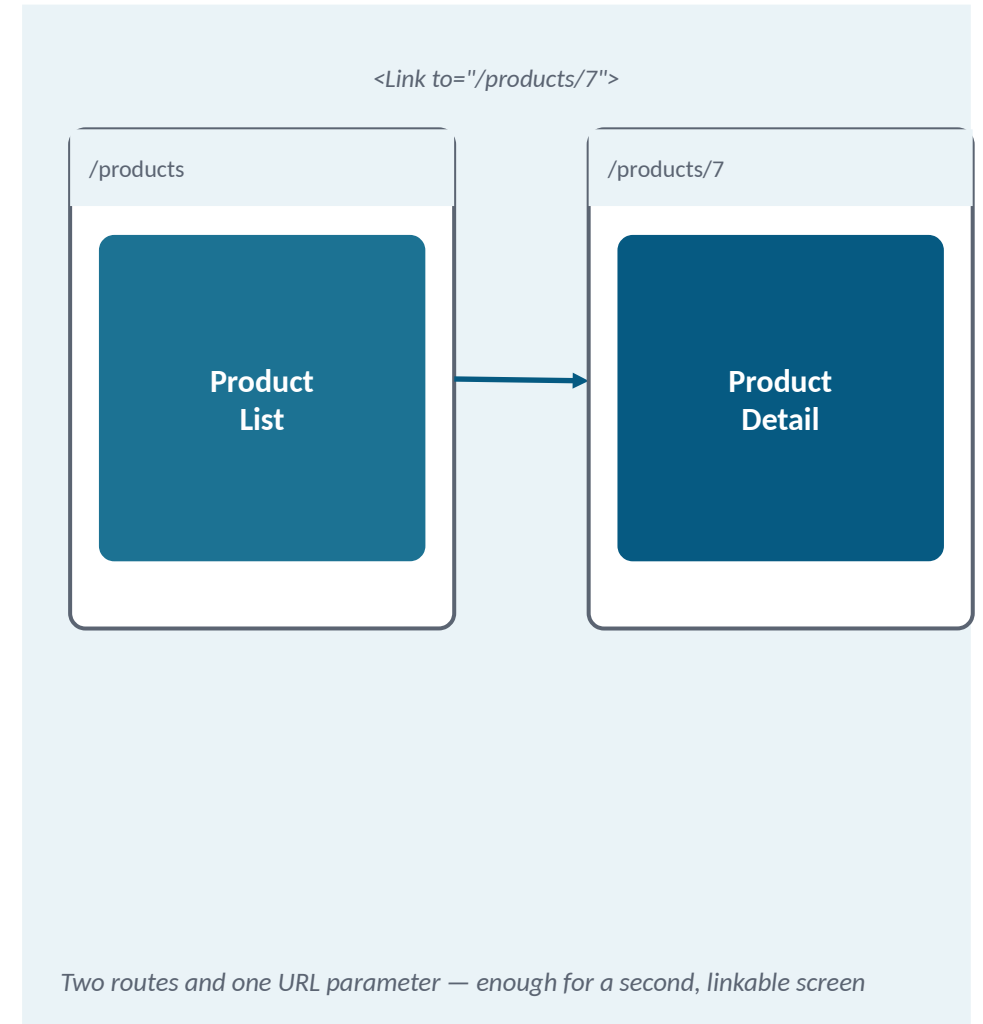
- React ships components, hooks, and rendering only — nothing else bundled in
- A framework (Next.js, Angular) bundles routing, data fetching, and server rendering as one package
- Adding one routing library next doesn't make this a framework
- Same spectrum, different points: library → routing library → full framework



Same spectrum, three different points — today adds the middle one

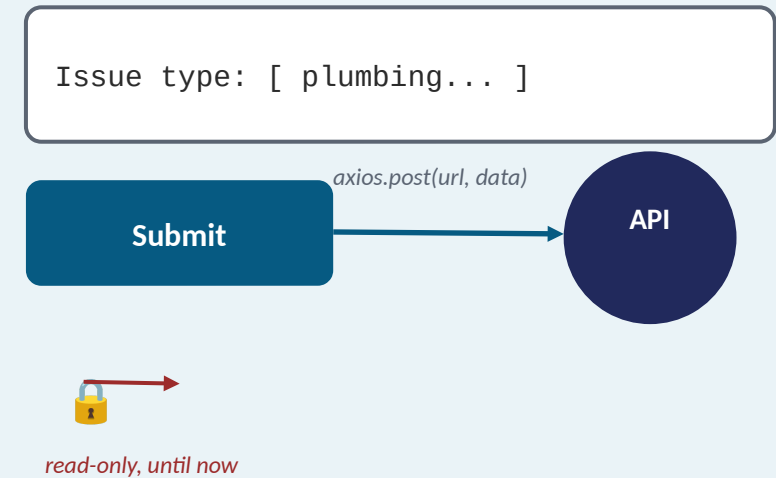
Routing Between Views

- React Router (npm install react-router) implements the 7 SPA characteristics
- `<BrowserRouter>` wraps the app; `<Routes>/<Route>` map a URL path to a component
- `<Link to="...">` changes the URL and swaps the view — no full-page reload
- `:productId` in a route path is a URL parameter, read with `useParams()`
- Full hands-on routing exercise is in today's lab guide



Writing Data Back: Forms & REST CRUD

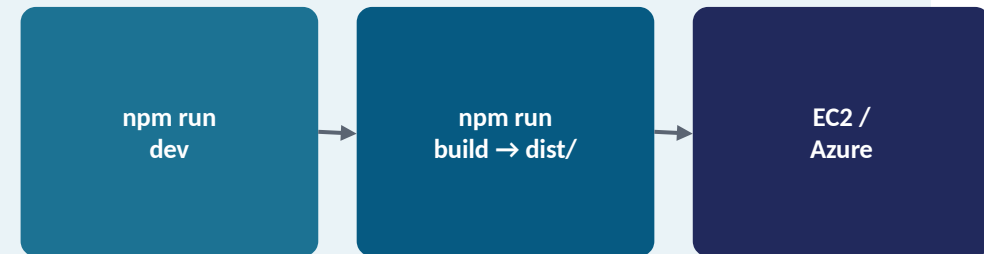
- Everything fetched today has been read-only — `axios.get` only reads data down
- A controlled input: `value={state} + onChange={e => setState(e.target.value)}`
- Writing back: `axios.post` (create), `.put` (update), `.delete` (remove)
- Blocks one team brief directly: a resident "submit maintenance requests" workflow
- Lab guide's Fake Store API is safe to practice on — writes never persist



A controlled form submitting a write via `axios.post` — the missing direction

From npm run dev to a Deployable Build

- npm run dev (today's server) is for editing — hot reload, unminified, never deployed
- npm run build compiles everything into a static dist/ folder
- npm run preview serves that exact build locally, to sanity-check it first
- dist/ is what the already-provisioned EC2/Azure setup actually serves
- Full hands-on build-and-deploy steps are in today's lab guide



npm run preview: local check first

One command, npm run build, is the missing link to the Final Sprint demo